

# FOODLOOP

## Schéma de Base de Données

Livrable L2 — Architecture BDD — Workshop BC04

|                  |                                                                     |
|------------------|---------------------------------------------------------------------|
| Document         | Schéma de base de données relationnel (ERD + SQL)                   |
| Auteur           | Erwan — Lead Dev Backend                                            |
| Version          | 1.0                                                                 |
| Date             | 19 juin 2026                                                        |
| Périmètre        | Application web FoodLoop — faces Producteur, Consommateur, Admin    |
| Référence projet | Workshop BC04 — Conception et développement d'une solution digitale |

### 1. Architecture générale

La base de données FoodLoop suit un modèle relationnel normalisé (3NF). Elle distingue trois types d'utilisateurs via une table **users** avec un champ **role** (consumer, producer, admin), et s'articule autour du cycle de vie d'une commande.

| Table A   | Card. | Table B       | Description                                         |
|-----------|-------|---------------|-----------------------------------------------------|
| users     | 1 → 1 | producers     | Un producteur est un utilisateur avec role=producer |
| producers | 1 → N | products      | Un producteur possède plusieurs produits            |
| users     | 1 → N | orders        | Un consommateur passe plusieurs commandes           |
| orders    | 1 → N | order_items   | Une commande contient plusieurs lignes              |
| products  | 1 → N | order_items   | Un produit apparaît dans plusieurs lignes           |
| hubs      | 1 → N | orders        | Un hub accueille plusieurs commandes                |
| users     | 1 → N | subscriptions | Un consommateur a plusieurs abonnements             |

### 2. Description des tables

#### 2.1 Table users

| Colonne                   | Type         | Contraintes                             | Description           |
|---------------------------|--------------|-----------------------------------------|-----------------------|
| id                        | UUID         | PK, DEFAULT<br>uuid_generate_v4()       | Identifiant unique    |
| email                     | VARCHAR(255) | UNIQUE, NOT NULL                        | Email de connexion    |
| password_hash             | VARCHAR(255) |                                         | Mot de passe bcrypt   |
| first_name /<br>last_name | VARCHAR(100) | NOT NULL                                | Prénom et nom         |
| role                      | VARCHAR(20)  | CHECK IN (consumer,<br>producer, admin) | Rôle de l'utilisateur |
| is_active                 | BOOLEAN      | DEFAULT true                            | Compte actif          |
| oauth_provider            | VARCHAR(50)  |                                         | Fournisseur OAuth     |
| created_at                | TIMESTAMP    | DEFAULT NOW()                           | Date de création      |

## 2.2 Table producers

| Colonne              | Type         | Contraintes                        | Description                       |
|----------------------|--------------|------------------------------------|-----------------------------------|
| id                   | UUID         | PK                                 | Identifiant unique                |
| user_id              | UUID         | FK → users(id), UNIQUE,<br>CASCADE | Référence utilisateur             |
| farm_name            | VARCHAR(255) | NOT NULL                           | Nom de la ferme                   |
| city                 | VARCHAR(100) |                                    | Ville                             |
| latitude / longitude | DECIMAL(9,6) |                                    | Coordonnées GPS                   |
| stripe_account_id    | VARCHAR(255) |                                    | ID compte Stripe Connect          |
| is_verified          | BOOLEAN      | DEFAULT false                      | Producteur vérifié                |
| labels               | TEXT[]       |                                    | Labels (bio, aop, label_rouge...) |

## 2.3 Table products

| Colonne         | Type          | Contraintes                    | Description                 |
|-----------------|---------------|--------------------------------|-----------------------------|
| id              | UUID          | PK                             | Identifiant unique          |
| producer_id     | UUID          | FK → producers(id),<br>CASCADE | Producteur propriétaire     |
| name            | VARCHAR(255)  | NOT NULL                       | Nom du produit              |
| price           | DECIMAL(10,2) | NOT NULL                       | Prix unitaire en euros      |
| unit            | VARCHAR(50)   | NOT NULL                       | Unité (kg, piece, litre...) |
| stock_quantity  | DECIMAL(10,2) | DEFAULT 0                      | Quantité en stock           |
| category        | VARCHAR(100)  |                                | Catégorie                   |
| labels / images | TEXT[]        |                                | Labels et URLs images       |
| is_available    | BOOLEAN       | DEFAULT true                   | Disponible à la vente       |

## 2.4 Table hubs

| Colonne       | Type         | Contraintes  | Description          |
|---------------|--------------|--------------|----------------------|
| id            | UUID         | PK           | Identifiant unique   |
| name          | VARCHAR(255) | NOT NULL     | Nom du hub           |
| address       | TEXT         | NOT NULL     | Adresse complète     |
| city          | VARCHAR(100) | NOT NULL     | Ville                |
| opening_hours | JSONB        |              | Horaires d'ouverture |
| is_active     | BOOLEAN      | DEFAULT true | Hub actif            |

## 2.5 Table orders

| Colonne      | Type          | Contraintes                                                                 | Description        |
|--------------|---------------|-----------------------------------------------------------------------------|--------------------|
| id           | UUID          | PK                                                                          | Identifiant unique |
| consumer_id  | UUID          | FK → users(id)                                                              | Consommateur       |
| hub_id       | UUID          | FK → hubs(id)                                                               | Point de retrait   |
| status       | VARCHAR(50)   | CHECK IN (pending,<br>confirmed, preparing, ready,<br>completed, cancelled) | Statut             |
| total_amount | DECIMAL(10,2) | NOT NULL                                                                    | Montant total      |

|                          |               |  |                          |
|--------------------------|---------------|--|--------------------------|
| platform_fee             | DECIMAL(10,2) |  | Commission FoodLoop (8%) |
| stripe_payment_intent_id | VARCHAR(255)  |  | ID PaymentIntent Stripe  |
| qr_code                  | VARCHAR(255)  |  | Code de retrait          |

## 2.6 Table order\_items

| Colonne     | Type          | Contraintes              | Description                   |
|-------------|---------------|--------------------------|-------------------------------|
| id          | UUID          | PK                       | Identifiant unique            |
| order_id    | UUID          | FK → orders(id), CASCADE | Commande parente              |
| product_id  | UUID          | FK → products(id)        | Produit commandé              |
| producer_id | UUID          | FK → producers(id)       | Producteur                    |
| quantity    | DECIMAL(10,2) | NOT NULL                 | Quantité commandée            |
| unit_price  | DECIMAL(10,2) | NOT NULL                 | Prix au moment de la commande |
| subtotal    | DECIMAL(10,2) | NOT NULL                 | quantity x unit_price         |

## 2.7 Table subscriptions

| Colonne         | Type    | Contraintes    | Description                    |
|-----------------|---------|----------------|--------------------------------|
| id              | UUID    | PK             | Identifiant unique             |
| consumer_id     | UUID    | FK → users(id) | Consommateur abonné            |
| hub_id          | UUID    | FK → hubs(id)  | Point de retrait               |
| items           | JSONB   | NOT NULL       | Liste [{product_id, quantity}] |
| is_active       | BOOLEAN | DEFAULT true   | Abonnement actif               |
| next_order_date | DATE    |                | Prochaine date de génération   |

### 3. Script SQL de création

---

Script complet d'initialisation de la base de données.

```
CREATE EXTENSION IF NOT EXISTS "uuid-oss";

CREATE TABLE IF NOT EXISTS users (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  email VARCHAR(255) UNIQUE NOT NULL, password_hash VARCHAR(255),
  first_name VARCHAR(100) NOT NULL, last_name VARCHAR(100) NOT NULL,
  role VARCHAR(20) NOT NULL CHECK (role IN ('consumer', 'producer', 'admin')),
  phone VARCHAR(20), is_active BOOLEAN DEFAULT true,
  created_at TIMESTAMP DEFAULT NOW()
);

CREATE TABLE IF NOT EXISTS producers (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  user_id UUID UNIQUE NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  farm_name VARCHAR(255) NOT NULL, city VARCHAR(100),
  latitude DECIMAL(9,6), longitude DECIMAL(9,6),
  stripe_account_id VARCHAR(255),
  is_verified BOOLEAN DEFAULT false, labels TEXT[],
  created_at TIMESTAMP DEFAULT NOW()
);

CREATE TABLE IF NOT EXISTS products (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  producer_id UUID NOT NULL REFERENCES producers(id) ON DELETE CASCADE,
  name VARCHAR(255) NOT NULL, price DECIMAL(10,2) NOT NULL,
  unit VARCHAR(50) NOT NULL, stock_quantity DECIMAL(10,2) DEFAULT 0,
  category VARCHAR(100), labels TEXT[], images TEXT[],
  is_available BOOLEAN DEFAULT true,
  created_at TIMESTAMP DEFAULT NOW(), updated_at TIMESTAMP DEFAULT NOW()
);

CREATE TABLE IF NOT EXISTS hubs (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  name VARCHAR(255) NOT NULL, address TEXT NOT NULL,
  city VARCHAR(100) NOT NULL, opening_hours JSONB,
  is_active BOOLEAN DEFAULT true, created_at TIMESTAMP DEFAULT NOW()
);

CREATE TABLE IF NOT EXISTS orders (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  consumer_id UUID NOT NULL REFERENCES users(id),
  hub_id UUID REFERENCES hubs(id),
  status VARCHAR(50) DEFAULT 'pending'
  CHECK (status IN ('pending', 'confirmed', 'preparing', 'ready', 'completed', 'cancelled')),
  total_amount DECIMAL(10,2) NOT NULL, platform_fee DECIMAL(10,2),
  stripe_payment_intent_id VARCHAR(255), qr_code VARCHAR(255),
  created_at TIMESTAMP DEFAULT NOW(), updated_at TIMESTAMP DEFAULT NOW()
);

CREATE TABLE IF NOT EXISTS order_items (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  order_id UUID NOT NULL REFERENCES orders(id) ON DELETE CASCADE,
  product_id UUID NOT NULL REFERENCES products(id),
  producer_id UUID NOT NULL REFERENCES producers(id),
  quantity DECIMAL(10,2) NOT NULL, unit_price DECIMAL(10,2) NOT NULL,
  subtotal DECIMAL(10,2) NOT NULL
```

```
);  
CREATE TABLE IF NOT EXISTS subscriptions (  
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
  consumer_id UUID NOT NULL REFERENCES users(id),  
  hub_id UUID REFERENCES hubs(id),  
  items JSONB NOT NULL, is_active BOOLEAN DEFAULT true,  
  next_order_date DATE, created_at TIMESTAMP DEFAULT NOW()  
);
```

## 4. Règles métier

| Règle         | Description                                                                                                   |
|---------------|---------------------------------------------------------------------------------------------------------------|
| Rôles         | Trois rôles : consumer, producer, admin. Un producteur ne peut pas passer de commande.                        |
| Commission    | FoodLoop prélève 8% sur chaque commande (platform_fee). Calculé côté serveur.                                 |
| Cycle statuts | pending → confirmed → preparing → ready → completed. Seul le consommateur confirme la récupération.           |
| Stock         | Décrémenté dans la même transaction SQL. ROLLBACK si insuffisant.                                             |
| Prix          | Le unit_price est copié dans order_items. Une modification ultérieure n'affecte pas les commandes existantes. |
| UUID v4       | Tous les IDs sont des UUID v4 générés par PostgreSQL via uuid-osp.                                            |